

Lập trình .NET (ASP.NET Core Web API)

**Buổi 2: Giới thiệu ASP.NET
Core Web API, kiến trúc
ứng dụng, Middleware,
Dependency Injection**



1. Lý thuyết

1.1. ASP.NET Core

1.2. Web API

1.3. Project Structure

1.4. Middleware

1.5. Dependency Injection

1.6. Program.cs

2. Thực hành

2.1. Tạo Web API

2.2. Chạy Swagger

2.3. Viết Controller đầu tiên

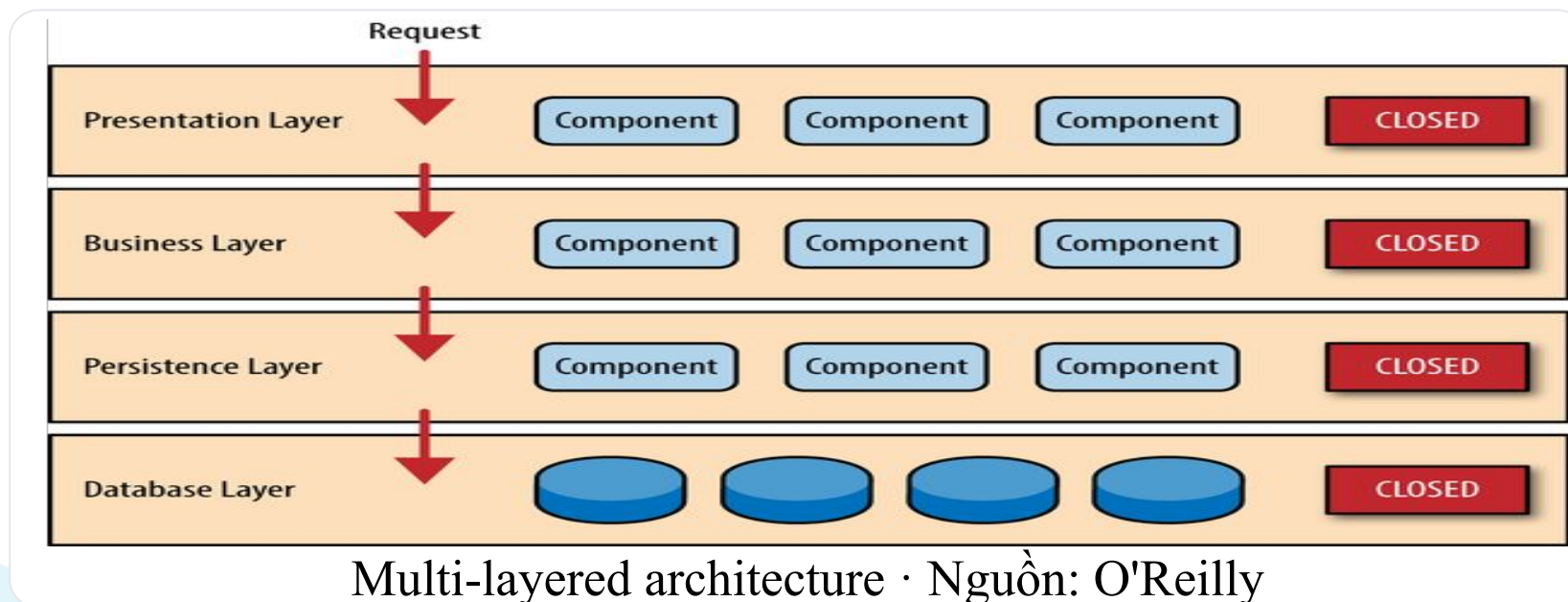
ASP.NET Core là gì?



1.1. ASP.NET Core (tt)

ASP.NET Core

- Framework xây dựng ứng dụng web hiện đại trên nền .NET
- Đa nền tảng: chạy trên Windows, Linux, macOS
- Hiệu năng cao, mã nguồn mở



Web API là gì?



1.2. Web API (tt)

Web API là gì?

- Dịch vụ HTTP trả về dữ liệu (JSON/XML), không trả giao diện HTML
- Frontend (React, Vue, Mobile) gọi API để lấy dữ liệu
- Các thành phần chính.



Khác biệt Web API vs MVC: Web API trả JSON, MVC trả HTML.

1.3. Project Structure

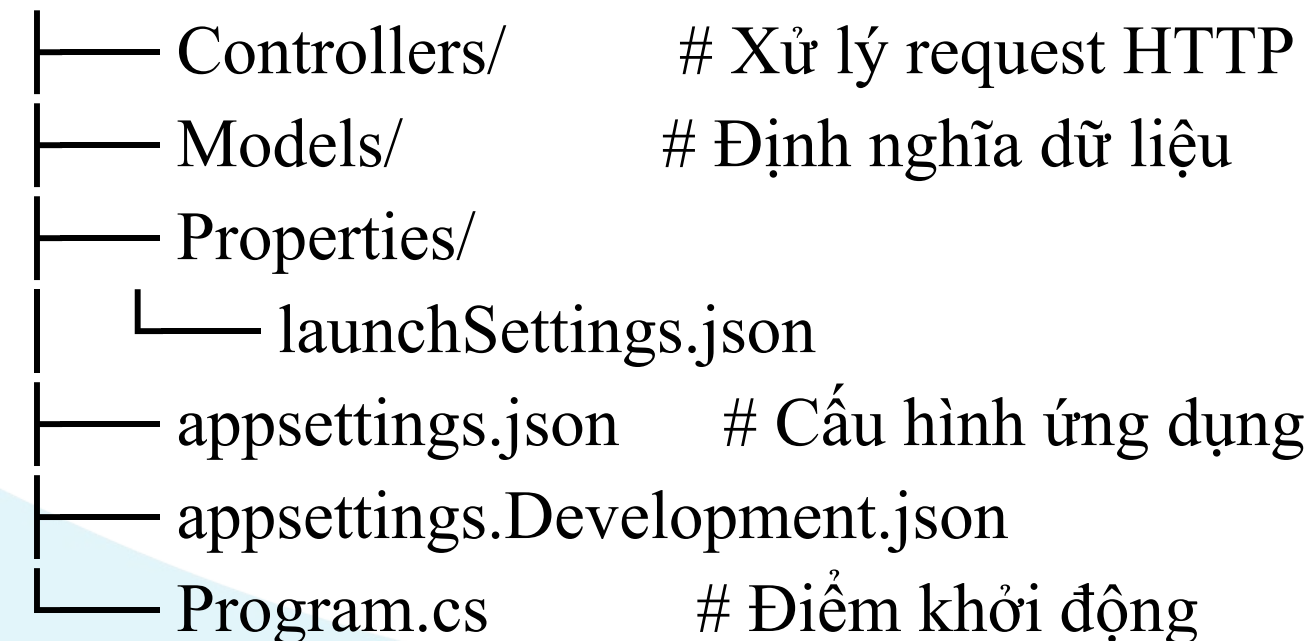
Tạo project với CLI

```
dotnet new webapi -n BookApi
```

```
cd BookApi
```

Cấu trúc thư mục tiêu chuẩn:

BookApi/



Ghi nhớ

Controllers/ là nơi nhận và xử lý request.
Models/ là nơi định nghĩa dữ liệu.

Điểm quan trọng

Program.cs là điểm khởi động duy nhất trong .NET 8 Minimal Hosting Model - không còn Startup.cs.

1.4. Middleware

Middleware là gì?



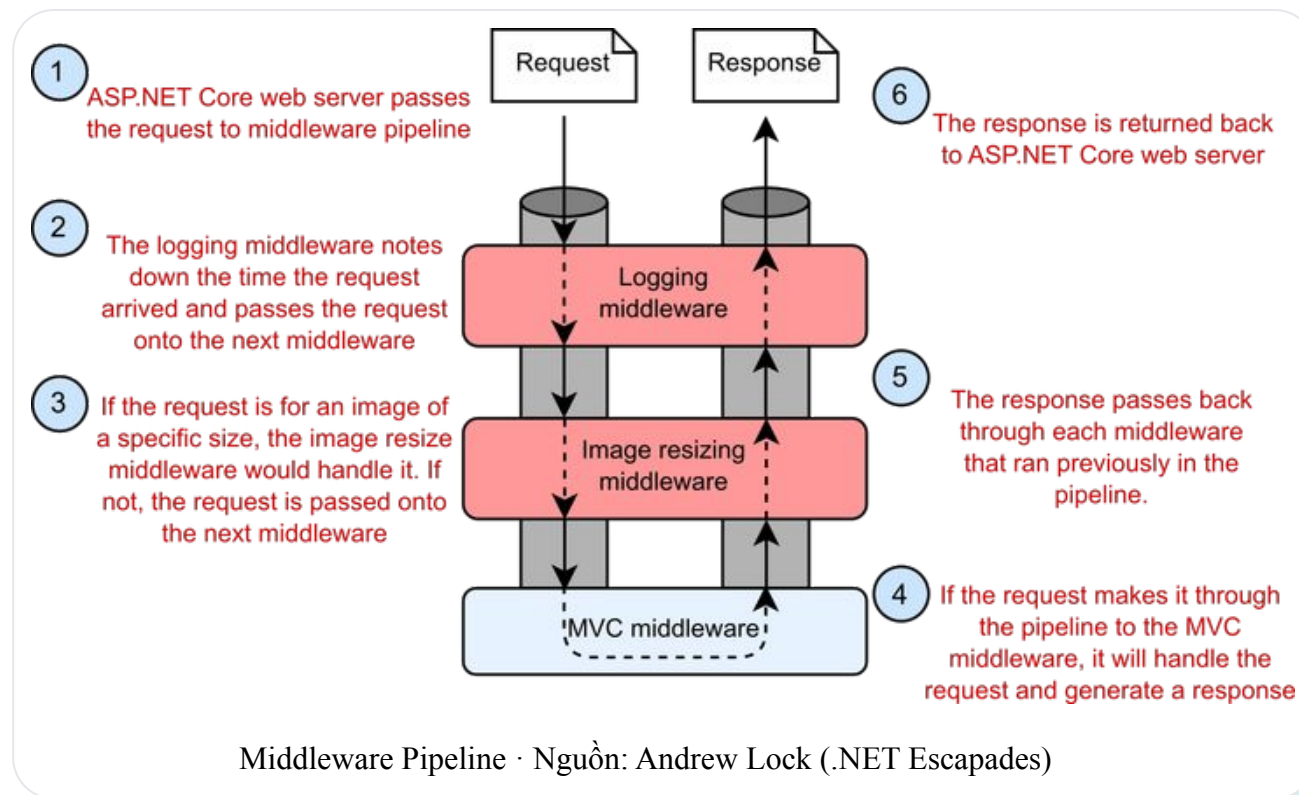
1.4. Middleware (tt)

Middleware là gì?

Là các thành phần xử lý HTTP request/response theo dạng pipeline. Mỗi middleware có thể: xử lý request, gọi middleware tiếp theo, hoặc dừng pipeline.

Middleware phổ biến

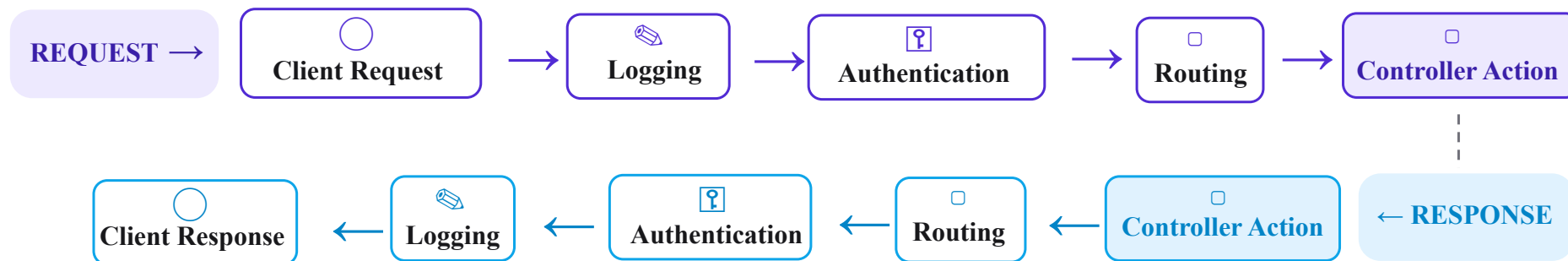
Middleware	Nhiệm vụ
UseHttpsRedirection	Chuyển HTTP → HTTPS
UseRouting	Xác định route cho request
UseAuthentication	Kiểm tra xác thực
UseAuthorization	Kiểm tra phân quyền
UseStaticFiles	Phục vụ file tĩnh



1.4. Middleware (tt)

Request Pipeline Hoạt Động Như Thế Nào?

Request đi vào theo thứ tự đăng ký → Response đi ra ngược lại



Lỗi thường gặp: Đăng ký sai thứ tự middleware. UseAuthentication() phải đặt TRƯỚC UseAuthorization(). Nếu sai → request chưa xác thực đã bị kiểm tra phân quyền → 403/401 sai.

1.5. Dependency Injection

Dependency Injection (DI) là gì?



1.5. Dependency Injection (tt)

Vấn đề:

// Tạo dependency trực tiếp bên trong class => Không nên

```
public class BookController
```

```
{
```

```
    private BookService _service = new BookService(); // Phụ thuộc cứng
```

```
}
```

1.5. Dependency Injection (tt)

DI giải quyết:

// Nhận dependency qua constructor

```
public class BookController : ControllerBase
{
    private readonly IBookService _service;
    public BookController(IBookService service)
    {
        _service = service; // DI Container tự inject
    }
}
```

1.5. Dependency Injection (tt)

Lợi ích:

- Dễ test (mock dependency)
- Dễ thay thế implementation
- Loose coupling

1.5. Dependency Injection (tt)

DI Container & Service Lifetime

3 cách đăng ký service trong Program.cs

```
builder.Services.AddSingleton<IMyService, MyService>();
```

```
builder.Services.AddScoped<IMyService, MyService>();
```

```
builder.Services.AddTransient<IMyService, MyService>();
```

1.5. Dependency Injection (tt)

DI Container & Service Lifetime

3 cách đăng ký service trong Program.cs

Lifetime	Tạo instance khi	Dùng cho
Singleton	Lần đầu tiên, dùng mãi	Config, Cache
Scoped	Mỗi HTTP request	DbContext, Repository
Transient	Mỗi lần inject	Stateless service

 **Lỗi thường gặp:** Inject “Scoped” vào “Singleton” → lỗi runtime.

 **Best Practice:** “DbContext” luôn dùng “AddDbContext()” (tương đương Scoped).

1.6. Program.cs

.NET 8 Minimal Hosting Model

```
var builder = WebApplication.CreateBuilder(args);
```

```
// 1. Đăng ký services (DI Container)
```

```
builder.Services.AddControllers();
```

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

```
var app = builder.Build();
```

1.6. Program.cs (tt)

.NET 8 Minimal Hosting Model

// 2. Cấu hình Middleware Pipeline

```
if (app.Environment.IsDevelopment())
```

```
{
```

```
    app.UseSwagger();
```

```
    app.UseSwaggerUI();
```

```
}
```

```
app.UseHttpsRedirection();
```

```
app.UseAuthorization();
```

```
app.MapControllers();
```

```
app.Run();
```



Ghi nhớ: “builder” để đăng ký services → “app” để cấu hình middleware → “Run()” để khởi chạy.

2.1. Tạo Web API

1. Tạo project

\$ dotnet new webapi -n BookApi

Hoặc VS: New Project → ASP.NET Core Web API

2. Chạy ứng dụng

\$ dotnet run

App lắng nghe ở <https://localhost:5001>

3. Mở Swagger

<https://localhost:5001/swagger>

Giao diện test API trên browser

4. Thêm BookController

Endpoint GET /api/book trả List<string>

Tạo file Controllers/BookController.cs

5. Test trên Swagger UI

Click "Try it out" → "Execute" → xem response JSON

Response 200 OK kèm body

◆ **Mẹo:** Đừng tắt app khi tes t. Chạy dotnet run trong terminal, mở Swagger ở browser khác. Hot reload: dotnet watch run.

2.2. Chạy Swagger

Swagger – Tài Liệu API Tự Động

Swagger là gì?

- Công cụ sinh tài liệu API từ code (OpenAPI spec)
- Giao diện web để test API ngay trên browser
- Đã được tích hợp sẵn trong template Web API (.NET 8)

Truy cập Swagger:

`https://localhost:{port}/swagger`



✓ Best Practice Luôn bật Swagger trong môi trường **Development** để test nhanh. Tắt hoặc bảo vệ trong **Production**.

2.2. Chạy Swagger (tt)

Swagger – Tài Liệu API Tự Động

```
1 ---
2 swagger: '2.0' #version of Swagger
3 info: # High level information of API
4   description: Sample Swagger Demo #Give the description of API
5   version: 1.0.0 #API version
6   title: Swagger Employee Details Demo # API title
7   license: #Swagger license info
8     name: Apache 2.0
9     url: 'http://www.apache.org/licenses/LICENSE-2.0.html'
10 host: localhost # Host Name
11 basePath: /v1 #Basepath when there are multiple versions of API running
12 tags: # Tag information for each API operation. Multiple tags for multiple API operation
13   - name: employee #tag name
14     description: Everything about your Employee #tag description of API operation
15   - name: health
16     description: Readiness probe
17 schemes:
18   - http #security schemes
19   - https
20 consumes:
21   - application/json #consumes content type
22 produces:
23   - application/json #produces content type
24 paths:
25   /employees:
26     post:
27       tags:
28         - employee
29       summary: Add a new employee
30       description: Add a new employees
31       operationId: addEmployees
32       parameters:
33         - in: body
34           name: body
35           description: Employees object that needs to be added
36           required: true
37           schema:
38             $ref: '#/definitions/Employee'
39       responses:
40         405:
41           description: Invalid input
42       '/employees/{employeeId}': #Request Mapping path of REST API
43         get: #Request method type, GET,POST etc.
44           tags: # Refer to created tag above
45             - employee
46           summary: Find employee by ID #Summary
47           description: Returns a single Employee #Description of API operation
48           operationId: getEmployeeDetails #Method name
49           produces:
50             - application/json #Response content type
51           parameters:
52             - name: employeeId #input parameter
```

Swagger Employee Details Demo 1.0.0

[Base URL: localhost/v1]

Sample Swagger Demo

Apache 2.0

Schemes

HTTP

employee Everything about your Employee

POST /employees Add a new employee

GET /employees/{employeeId} Find employee by ID

PUT /employees/{employeeId} Update employee by ID

DELETE /employees/{employeeId} Deletes employee by ID

health Readiness probe

GET /health

Models

Swagger UI · Giao diện test API trực tiếp trên browser

2.3. Viết Controller đầu tiên

Tạo “**Controllers/BookController.cs**”:

```
using Microsoft.AspNetCore.Mvc;
namespace BookApi.Controllers;
[ApiController]
[Route("api/[controller]")]
public class BookController : ControllerBase
{
    [HttpGet]
    public IActionResult GetAll()
    {
        var books = new List<string> { "Clean Code", "DDD", "C# in Depth" };
        return Ok(books);
    }
}
```

2.3. Viết Controller đầu tiên (tt)

Giải thích:

Thành phần	Ý nghĩa
[ApiController]	Tự động validate model, trả lỗi 400
[Route("api/[controller]")]	Route: `/api/book`
ControllerBase	Base class cho API (không có View)
Ok(books)	Trả 200 OK kèm dữ liệu

2.4. Thêm Service với DI

Tạo Interface và Service:

// Interfaces/IBookService.cs

```
public interface IBookService  
{  
    List<string> GetAll();  
}
```

// Services/BookService.cs

```
public class BookService : IBookService  
{  
    public List<string> GetAll()  
        => new List<string> { "Clean Code", "DDD", "C# in Depth" };  
}
```

2.4. Thêm Service với DI (tt)

Đăng ký trong Program.cs:

```
builder.Services.AddScoped<IBookService, BookService>();
```

Inject vào Controller:

```
public class BookController : ControllerBase
{
    private readonly IBookService _bookService;

    public BookController(IBookService bookService)
    {
        _bookService = bookService;
    }

    [HttpGet]
    public IActionResult GetAll() => Ok(_bookService.GetAll());
}
```

2.5. Bài thực hành

1. ASP.NET Core Web API tên ProductApi. Chạy thành công và mở Swagger UI trên browser.
2. Endpoint GET /api/product. Trả về List<string> 3 tên sản phẩm. Kiểm tra bằng Swagger.
3. Interface với method GetAll(). ProductService implement interface. Đăng ký AddScoped, inject vào ProductController.
4. In console: [LOG] Request: {Method} {Path} cho mỗi request. Đăng ký bằng app.Use() trong Program.cs.
5. Tạo 3 service (Singleton/Scoped/Transient). Mỗi service có property Id = Guid.NewGuid(). Gọi 2 lần trong cùng request, in Id → quan sát khác nhau.



Thank @ You!